30.1 内部钩子（Gateway Hooks）

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~120k tokens，输出 ~8k tokens（本节）

OpenClaw 的钩子系统（Hook System）是一套事件驱动的扩展机制，允许在 Agent 生命周期的关键节点注入自定义逻辑——无需修改核心代码。本节聚焦**内部钩子**（Internal Hooks），即运行在 Gateway 进程内部、通过事件注册/触发的钩子体系。

> **衍生解释**：钩子（Hook）是软件工程中的经典模式。它在程序执行流的某些"节点"预留了扩展点，让外部代码可以在这些节点上"挂钩"自己的逻辑。React 的生命周期方法、Git 的 pre-commit/post-commit、Webpack 的 tapable 插件系统，都是钩子模式的不同实现。OpenClaw 的钩子采用的是**发布/订阅（Pub/Sub）变体**——注册处理器监听事件键，事件触发时按注册顺序依次调用。

30.1.1 钩子加载器（`src/hooks/loader.ts`）

加载流程概览

`loadInternalHooks` 是钩子系统的入口函数。它在 Gateway 启动时被调用，负责发现、过滤、导入并注册所有内部钩子处理器。

```typescript
// src/hooks/loader.ts（简化）

export async function loadInternalHooks(
  cfg: OpenClawConfig,
  workspaceDir: string,
): Promise<number> {
  // 前置检查：钩子系统是否启用
  if (!cfg.hooks?.internal?.enabled) {
    return 0;
  }

  let loadedCount = 0;

  // 阶段一：从目录发现钩子（新系统）
  const hookEntries = loadWorkspaceHookEntries(workspaceDir, { config: cfg });
  const eligible = hookEntries.filter(entry =>
    shouldIncludeHook({ entry, config: cfg })
  );

  for (const entry of eligible) {
    // 跳过配置中显式禁用的钩子
    const hookConfig = resolveHookConfig(cfg, entry.hook.name);
    if (hookConfig?.enabled === false) continue;

    // 动态导入处理器模块（带缓存清除）
    const url = pathToFileURL(entry.hook.handlerPath).href;
    const mod = await import(`${url}?t=${Date.now()}`);

    // 获取导出的处理器函数
    const exportName = entry.metadata?.export ?? "default";
    const handler = mod[exportName];

    // 为元数据中声明的每个事件注册处理器
    const events = entry.metadata?.events ?? [];
    for (const event of events) {
      registerInternalHook(event, handler);
    }
    loadedCount++;
  }

  // 阶段二：加载遗留配置处理器（向后兼容）
  const handlers = cfg.hooks.internal.handlers ?? [];
  for (const handlerConfig of handlers) {
```

```
    const modulePath = path.isAbsolute(handlerConfig.module)
      ? handlerConfig.module
      : path.join(process.cwd(), handlerConfig.module);
    const mod = await import(pathToFileURL(modulePath).href);
    registerInternalHook(handlerConfig.event, mod[handlerConfig.export ?? "default"]);
    loadedCount++;
  }

  return loadedCount;
}
```

整个加载流程分两个阶段：

阶段	来源	机制	说明
阶段一	目录发现	`loadWorkspaceHookEntries` → `shouldIncludeHook` 过滤 → 动态 `import()`	新系统，基于 HOOK.md 文件发现
阶段二	配置文件	`cfg.hooks.internal.handlers` 数组	遗留系统，向后兼容

这里有两个细节值得留意：

1. **缓存清除（Cache Busting）**：动态导入的 URL 后附加了 `?t=${Date.now()}` 时间戳参数，确保每次加载都获取最新版本的处理器代码，不会被 Node.js 的模块缓存影响。
2. **双重禁用检查**：先通过 `shouldIncludeHook` 做资格过滤，再检查 `hookConfig?.enabled === false` 做配置级禁用。两道关卡确保只有合格且启用的钩子才会被加载。

钩子发现：四源合并

钩子的发现与技能系统（第 21 章）类似，从四个目录源加载并合并：

```ts
// src/hooks/workspace.ts（简化）

function loadHookEntries(workspaceDir: string, opts?): HookEntry[] {
  const bundledHooksDir = opts?.bundledHooksDir ?? resolveBundledHooksDir();
  const managedHooksDir = opts?.managedHooksDir ?? path.join(CONFIG_DIR, "hooks");
  const workspaceHooksDir = path.join(workspaceDir, "hooks");
  const extraDirs = opts?.config?.hooks?.internal?.load?.extraDirs ?? [];

  // 按优先级从低到高加载：extra < bundled < managed < workspace
  const merged = new Map<string, Hook>();
  for (const hook of extraHooks)    merged.set(hook.name, hook);
  for (const hook of bundledHooks)  merged.set(hook.name, hook);
  for (const hook of managedHooks)  merged.set(hook.name, hook);
  for (const hook of workspaceHooks) merged.set(hook.name, hook);

  return Array.from(merged.values()).map(hook => ({
    hook,
    frontmatter: parseFrontmatter(readHookMd(hook.filePath)),
    metadata: resolveOpenClawMetadata(frontmatter),
    invocation: resolveHookInvocationPolicy(frontmatter),
  }));
}
```

来源	目录	来源标识	优先级
Extra	配置的额外目录	`openclaw-workspace`	最低
Bundled	`<package>/src/hooks/bundled/`	`openclaw-bundled`	低
Managed	`~/.openclaw/hooks/`	`openclaw-managed`	高
Workspace	`<workspace>/hooks/`	`openclaw-workspace`	最高

同名钩子遵循**高优先级覆盖低优先级**原则——如果工作区有一个 `session-memory` 钩子，它会覆盖内置的同名钩子。

钩子文件结构

每个钩子是一个目录，至少包含两个文件：

```
my-hook/
├── HOOK.md          # 钩子描述文件（Front Matter + 文档）
└── handler.ts       # 钩子处理器（默认导出一个函数）
```

`HOOK.md` 的 Front Matter 定义了钩子的元数据：

```
---
name: session-memory
description: "Save session context to memory when /new command is issued"
metadata:
  {
    "openclaw":
      {
        "emoji": "...",
        "events": ["command:new"],
        "requires": { "config": ["workspace.dir"] },
        "install": [{ "id": "bundled", "kind": "bundled" }],
      },
  }
---
```

`resolveOpenClawMetadata` 负责解析 Front Matter 中 `metadata` 字段下的 `openclaw` 对象，提取出 `OpenClawHookMetadata`：

```typescr
// src/hooks/types.ts

export type OpenClawHookMetadata = {
  always?: boolean;          // 是否绕过所有前置检查
  hookKey?: string;          // 配置键名（默认为钩子名称）
  emoji?: string;            // 显示图标
  events: string[];          // 订阅的事件列表，如 ["command:new"]
  export?: string;           // 处理器导出名（默认 "default"）
  os?: string[];             // 操作系统要求
  requires?: {
    bins?: string[];         // 必须存在的二进制命令
    anyBins?: string[];      // 至少有一个的命令
    env?: string[];          // 必须存在的环境变量
    config?: string[];       // 必须为真的配置路径
  };
  install?: HookInstallSpec[];  // 安装选项
};
```

资格过滤

`shouldIncludeHook` 对每个钩子进行多维度的资格检查：

```
// src/hooks/config.ts（简化）

export function shouldIncludeHook(params: {
  entry: HookEntry;
  config?: OpenClawConfig;
}): boolean {
  const { entry, config } = params;

  // 1. 插件托管的钩子不受用户配置控制
  // 2. 配置中显式禁用 → 排除
  if (!pluginManaged && hookConfig?.enabled === false) return false;

  // 3. 操作系统不匹配 → 排除
  if (osList.length > 0 && !osList.includes(process.platform)) return false;

  // 4. always 标记 → 绕过后续所有检查
  if (entry.metadata?.always === true) return true;

  // 5. 必须的二进制命令全部存在?
  for (const bin of requiredBins) {
    if (!hasBinary(bin)) return false;
  }

  // 6. 至少一个可选命令存在?
  if (requiredAnyBins.length > 0 && !requiredAnyBins.some(hasBinary)) return false;

  // 7. 必须的环境变量存在? （配置中的 env 也算）
  for (const envName of requiredEnv) {
    if (!process.env[envName] && !hookConfig?.env?.[envName]) return false;
  }

  // 8. 必须的配置路径为真值?
  for (const configPath of requiredConfig) {
    if (!isConfigPathTruthy(config, configPath)) return false;
  }

  return true;
}
```

这套过滤机制和技能系统的 `shouldIncludeSkill`（第 21.1 节）高度对称——相同的设计哲学，钩子能用的检查维度和技能一致。钩子可以声明"我只在 macOS 上运行"、"我需要 `ffmpeg` 命令"、"我需要 `OPENAI_API_KEY` 环境变量"等前提条件。

30.1.2 钩子安装（`src/hooks/install.ts`）

安装来源

OpenClaw 支持三种钩子安装来源：

入口函数	来源	工作方式
`installHooksFromPath`	本地路径	目录 → 直接复制；归档 → 解压后复制
`installHooksFromArchive`	归档文件	`.tar.gz` / `.zip` → 临时目录解压 → 复制
`installHooksFromNpmSpec`	npm 包	`npm pack` 下载 → 作为归档安装

所有路径最终汇聚到两个核心函数：

```
installHooksFromPath ──┬──→ installHookPackageFromDir（有 package.json 的钩子包）
                       └──→ installHookFromDir（单个钩子目录）

installHooksFromArchive ──→ 解压 ──→ 同上两路径
installHooksFromNpmSpec ──→ npm pack ──→ installHooksFromArchive
```

钩子包 vs 单个钩子

OpenClaw 区分两种安装粒度：

单个钩子——一个包含 `HOOK.md` + `handler.ts` 的目录：

```
my-hook/
├── HOOK.md
└── handler.ts
```

钩子包——一个 npm 风格的包，可包含多个钩子：

```
my-hook-pack/
├── package.json        # 包含 "openclaw": { "hooks": ["./hook-a", "./hook-b"] }
├── hook-a/
│   ├── HOOK.md
│   └── handler.ts
└── hook-b/
    ├── HOOK.md
    └── handler.ts
```

package.json 中通过 openclaw.hooks 数组声明包含的钩子子目录：

```
{
  "name": "my-hook-pack",
  "version": "1.0.0",
  "openclaw": {
    "hooks": ["./hook-a", "./hook-b"]
  }
}
```

安装流程

以 installHookPackageFromDir 为例，安装一个钩子包的完整流程：

```typescript
// src/hooks/install.ts（简化）

async function installHookPackageFromDir(params): Promise<InstallHooksResult> {
  // 1. 读取并验证 package.json
  const manifest = await readJsonFile<HookPackageManifest>(manifestPath);
  const hookEntries = await ensureOpenClawHooks(manifest);

  // 2. 确定钩子包 ID（取 package.json 的 name，去掉 scope）
  const hookPackId = unscopedPackageName(manifest.name) || path.basename(packageDir);

  // 3. 解析安全的安装目标路径（防路径穿越）
  const targetDir = resolveSafeInstallDir(hooksDir, hookPackId);

  // 4. 验证所有子钩子目录（HOOK.md + handler 存在）
  for (const entry of hookEntries) {
    await validateHookDir(path.resolve(packageDir, entry));
  }

  // 5. 如果是更新模式，先备份旧版本
  if (mode === "update" && await fileExists(targetDir)) {
    backupDir = `${targetDir}.backup-${Date.now()}`;
    await fs.rename(targetDir, backupDir);
  }

  // 6. 复制到目标目录
  await fs.cp(packageDir, targetDir, { recursive: true });

  // 7. 安装 npm 依赖（如果有）
  if (Object.keys(manifest.dependencies ?? {}).length > 0) {
    await runCommandWithTimeout(["npm", "install", "--omit=dev", "--silent"], {
      cwd: targetDir,
    });
  }

  // 8. 清理备份
  if (backupDir) await fs.rm(backupDir, { recursive: true, force: true });

  return { ok: true, hookPackId, hooks: resolvedHooks, targetDir };
}
```

路径穿越防护

安装路径的计算中内置了路径穿越（Path Traversal）检测：

```
// src/hooks/install.ts

function resolveSafeInstallDir(hooksDir: string, hookId: string) {
  const targetDir = path.join(hooksDir, safeDirName(hookId));
  const resolvedBase = path.resolve(hooksDir);
  const resolvedTarget = path.resolve(targetDir);
  const relative = path.relative(resolvedBase, resolvedTarget);

  // 如果相对路径以 ".." 开头或是绝对路径，说明存在穿越
  if (!relative || relative === ".." ||
      relative.startsWith(`..${path.sep}`) ||
      path.isAbsolute(relative)) {
    return { ok: false, error: "path traversal detected" };
  }
  return { ok: true, path: targetDir };
}
```

> **衍生解释**：路径穿越（Path Traversal）是一种常见的安全漏洞，攻击者通过构造包含 `../` 的路径来访问目标目录之外的文件。例如，钩子名称为 `../../etc` 时，拼接后可能指向系统目录。OpenClaw 通过 `path.relative()` 计算相对路径，确保目标始终位于钩子安装基础目录内。

更新与回滚

钩子更新采用**备份-替换-清理**策略：

1. 如果目标目录已存在，先重命名为 `<dir>.backup-<timestamp>`
2. 复制新版本到目标目录
3. 如果复制或 `npm install` 失败，自动从备份恢复
4. 成功后删除备份

这种策略保证了更新的原子性——不会出现"旧版本已删除、新版本安装失败"的中间状态。

30.1.3 内置钩子（`src/hooks/bundled/`）

OpenClaw 内置了四个钩子，覆盖了从启动初始化到会话管理的典型场景：

钩子名称	订阅事件	功能	前置条件
`boot-md`	`gateway:startup`	Gateway 启动时运行 `BOOT.md` 引导清单	`workspace.dir` 配置
`session-memory`	`command:new`	执行 `/new` 时保存会话到记忆文件	`workspace.dir` 配置
`command-logger`	`command`	记录所有命令事件到审计日志	无
`soul-evil`	`agent:bootstrap`	在特定条件下替换 SOUL.md 为 SOUL_EVIL.md	显式启用配置

boot-md：启动引导

`boot-md` 是最简单的内置钩子——它监听 `gateway:startup` 事件，在 Gateway 启动时自动运行工作区中的 `BOOT.md` 引导文件：

```
// src/hooks/bundled/boot-md/handler.ts

const runBootChecklist: HookHandler = async (event) => {
  if (event.type !== "gateway" || event.action !== "startup") return;

  const context = event.context as BootHookContext;
  if (!context.cfg || !context.workspaceDir) return;

  // 执行 BOOT.md 引导清单
  const deps = context.deps ?? createDefaultDeps();
  await runBootOnce({ cfg: context.cfg, deps, workspaceDir: context.workspaceDir });
};
```

这个钩子的价值在于自动化——用户可以在工作区放置一个 `BOOT.md` 文件，定义 Gateway 启动时需要执行的准备工作（如检查依赖、初始化数据库、拉取最新配置等），每次 Gateway 重启都会自动执行。

session-memory：会话记忆保存

`session-memory` 是四个内置钩子中最复杂的。它在用户执行 `/new` 命令（开始新会话）时，将**即将被丢弃的旧会话**内容保存到记忆目录：

```
// src/hooks/bundled/session-memory/handler.ts（简化）

const saveSessionToMemory: HookHandler = async (event) => {
  if (event.type !== "command" || event.action !== "new") return;

  const cfg = event.context.cfg as OpenClawConfig;
  const workspaceDir = resolveAgentWorkspaceDir(cfg, agentId);
  const memoryDir = path.join(workspaceDir, "memory");

  // 1. 读取会话文件中最近的 N 条消息
  const messageCount = hookConfig?.messages ?? 15;
  const sessionContent = await getRecentSessionContent(sessionFile, messageCount);

  // 2. 使用 LLM 生成描述性文件名
  let slug = await generateSlugViaLLM({ sessionContent, cfg });
  if (!slug) slug = formatTimeSlug(event.timestamp);  // 回退到时间戳

  // 3. 写入 Markdown 记忆文件
  const filename = `${dateStr}-${slug}.md`;  // 如 2026-01-16-api-design.md
  await fs.writeFile(path.join(memoryDir, filename), buildMarkdownEntry());
};
```

流程分三步：

1. **提取会话内容**：从 JSONL 格式的会话文件中解析最近 N 条 user/assistant 消息（跳过以 `/` 开头的命令消息）。
2. **LLM 生成文件名**：调用配置好的 LLM 提供者，基于会话内容生成一个 1-2 个单词的文件名别名（slug），如 `vendor-pitch` 、 `api-design` 。这通过 `generateSlugViaLLM` 函数实现，它会启动一个临时的嵌入式 Agent 来完成这个小任务。如果 LLM 调用失败或在测试环境中，则回退到时间戳格式 `HHMM` 。
3. **保存记忆文件**：在 `<workspace>/memory/` 目录下创建 `YYYY-MM-DD-slug.md` 文件，内容包含会话键、会话 ID、来源信息和对话摘要。

command-logger：命令审计日志

`command-logger` 订阅所有 `command` 类型事件（注意不是特定的 `command:new` ，而是通配的 `command` ），将每次命令执行以 JSONL 格式追加到日志文件：

```
// src/hooks/bundled/command-logger/handler.ts（简化）

const logCommand: HookHandler = async (event) => {
  if (event.type !== "command") return;

  const logFile = path.join(stateDir, "logs", "commands.log");
  const logLine = JSON.stringify({
    timestamp: event.timestamp.toISOString(),
    action: event.action,         // "new", "reset", "stop" 等
    sessionKey: event.sessionKey,
    senderId: event.context.senderId ?? "unknown",
    source: event.context.commandSource ?? "unknown",
  }) + "\n";

  await fs.appendFile(logFile, logLine, "utf-8");
};
```

日志文件位于 `~/.openclaw/logs/commands.log`，每行是一个独立的 JSON 对象，方便用 `jq` 等工具分析：

```
{"timestamp":"2026-01-16T14:30:00.000Z","action":"new","sessionKey":"agent:main:main","
```

soul-evil：人格替换

`soul-evil` 是个有点意思的钩子——它可以在特定条件下将 Agent 的系统人格文件 `SOUL.md` 替换为 `SOUL_EVIL.md`，让 Agent 表现出完全不同的"人格"。

```
// src/hooks/bundled/soul-evil/handler.ts（简化）

const soulEvilHook: HookHandler = async (event) => {
  if (!isAgentBootstrapEvent(event)) return;

  // 子 Agent 不受影响
  if (isSubagentSessionKey(context.sessionKey)) return;

  // 读取钩子配置
  const soulConfig = resolveSoulEvilConfigFromHook(hookConfig);
  if (!soulConfig) return;

  // 执行替换决策并更新引导文件
  const updated = await applySoulEvilOverride({
    files: context.bootstrapFiles,
    workspaceDir,
    config: soulConfig,
  });
  context.bootstrapFiles = updated;
};
```

触发条件有两种：

1. **净化窗口（Purge Window）**：配置每天一个固定时间段（如 21:00-21:15），在此期间所有请求使用 SOUL_EVIL.md。
2. **随机概率**：配置 0-1 之间的概率值，每次请求按概率随机触发。

这个钩子默认**不启用**——它的元数据中声明了 `requires.config: ["hooks.internal.entries.soul-evil.enabled"]`，只有用户显式在配置中设置 `enabled: true` 才会激活。

30.1.4 `agent:bootstrap` 钩子

`agent:bootstrap` 是内部钩子系统中最强大的事件——它在 Agent 构建系统提示词之前触发，允许钩子修改 Agent 的引导文件列表。

事件结构

```
// src/hooks/internal-hooks.ts

export type AgentBootstrapHookContext = {
  workspaceDir: string;                    // 工作区目录
  bootstrapFiles: WorkspaceBootstrapFile[]; // 引导文件列表（可修改）
  cfg?: OpenClawConfig;                    // 全局配置
  sessionKey?: string;                     // 会话键
  sessionId?: string;                      // 会话 ID
  agentId?: string;                        // Agent ID
};

export type AgentBootstrapHookEvent = InternalHookEvent & {
  type: "agent";
  action: "bootstrap";
  context: AgentBootstrapHookContext;
};
```

和其他事件不同， `agent:bootstrap` 的 `context` 中包含了 `bootstrapFiles` 数组——这是 Agent 构建系统提示词时使用的工作区文件列表（如 `SOUL.md` 、 `AGENTS.md` 、 `BOOT.md` 等）。钩子可以**直接修改**这个数组，从而影响 Agent 的行为。

类型守卫

由于 `InternalHookEvent.context` 的类型是宽泛的 `Record<string, unknown>` ，OpenClaw 提供了类型守卫函数来安全地收窄类型：

```
// src/hooks/internal-hooks.ts

export function isAgentBootstrapEvent(
  event: InternalHookEvent
): event is AgentBootstrapHookEvent {
  if (event.type !== "agent" || event.action !== "bootstrap") return false;
  const context = event.context as Partial<AgentBootstrapHookContext>;
  if (typeof context.workspaceDir !== "string") return false;
  return Array.isArray(context.bootstrapFiles);
}
```

> **衍生解释**：TypeScript 的**类型守卫（Type Guard）是一种通过运行时检查来收窄类型的机制。 `event is AgentBootstrapHookEvent` 是一个类型谓词（Type Predicate）**——当函数返回

> true 时，TypeScript 编译器会将 event 的类型从 InternalHookEvent 收窄为 AgentBootstrapHookEvent ，从而允许安全地访问 context.bootstrapFiles 等字段。

实际应用：soul-evil 的引导文件替换

soul-evil 钩子是 agent:bootstrap 事件的典型消费者。它的核心逻辑在 applySoulEvilOverride 中：

```
// src/hooks/soul-evil.ts（简化）

export async function applySoulEvilOverride(params: {
  files: WorkspaceBootstrapFile[];
  workspaceDir: string;
  config?: SoulEvilConfig;
}): Promise<WorkspaceBootstrapFile[]> {
  // 决策：是否使用 evil 人格
  const decision = decideSoulEvil({ config: params.config });
  if (!decision.useEvil) return params.files;

  // 读取 SOUL_EVIL.md 内容
  const evilPath = path.join(params.workspaceDir, decision.fileName);
  const evilContent = await fs.readFile(evilPath, "utf-8");

  // 替换引导文件列表中的 SOUL.md 内容（不修改磁盘文件）
  return params.files.map(file =>
    file.name === "SOUL.md"
      ? { ...file, content: evilContent, missing: false }
      : file
  );
}
```

注意这里的关键设计：钩子**不修改磁盘上的文件**，只是替换了内存中引导文件列表里 SOUL.md 条目的 content 字段。替换是临时的——只影响当前请求的系统提示词构建，不会永久改变文件。

30.1.5 命令钩子： /new 、 /reset 、 /stop

事件模型

内部钩子的事件系统基于一个简洁的二维模型：**类型（type）+ 动作（action）**。

```typescript
// src/hooks/internal-hooks.ts

export type InternalHookEventType = "command" | "session" | "agent" | "gateway";

export interface InternalHookEvent {
  type: InternalHookEventType;    // 事件类型
  action: string;                  // 具体动作
  sessionKey: string;              // 关联的会话键
  context: Record<string, unknown>; // 附加上下文
  timestamp: Date;                 // 时间戳
  messages: string[];              // 钩子可回推的消息
}
```

注册与触发

注册时可以订阅两种粒度的事件键：

```typescript
// 订阅所有命令事件（command:new、command:reset、command:stop 等）
registerInternalHook("command", myHandler);

// 只订阅 /new 命令
registerInternalHook("command:new", myHandler);
```

触发时，系统会同时调用两种订阅的处理器：

```
// src/hooks/internal-hooks.ts

export async function triggerInternalHook(event: InternalHookEvent): Promise<void> {
  // 获取类型级别的处理器（如 "command"）
  const typeHandlers = handlers.get(event.type) ?? [];
  // 获取动作级别的处理器（如 "command:new"）
  const specificHandlers = handlers.get(`${event.type}:${event.action}`) ?? [];

  // 合并后按注册顺序依次执行
  for (const handler of [...typeHandlers, ...specificHandlers]) {
    try {
      await handler(event);
    } catch (err) {
      // 错误隔离：一个钩子失败不影响其他钩子
      console.error(`Hook error [${event.type}:${event.action}]:`, err.message);
    }
  }
}
```

这种设计有两个重要特性：

1. **双层订阅**： `command` 可以捕获所有命令事件（如 `command-logger` 的审计需求）， `command:new` 只捕获特定命令（如 `session-memory` 的精确触发）。
2. **错误隔离**：每个处理器的执行被 try/catch 包裹，一个钩子抛出异常不会中断其他钩子的执行。这对生产环境很重要——不能因为一个第三方钩子的 bug 导致整个命令处理流程崩溃。

命令事件的生命周期

当用户执行 `/new` 命令时，系统会创建并触发一个命令事件：

```
// 系统内部调用（简化）
const event = createInternalHookEvent(
  "command",     // type
  "new",         // action
  sessionKey,    // sessionKey
  {
    cfg,
    previousSessionEntry,  // 前一个会话的信息
    commandSource: "telegram",
    senderId: "+1234567890",
  }
);

await triggerInternalHook(event);
```

对于这个事件，以下钩子会被触发：

钩子	订阅键	匹配方式
`command-logger`	`command`	类型匹配（所有命令都会触发）
`session-memory`	`command:new`	动作匹配（只有 `/new` 触发）

消息回推

`InternalHookEvent` 中的 `messages` 数组是一个双向通信机制——钩子可以向这个数组推送消息，这些消息会被返回给用户：

```
// 钩子处理器可以回推消息
const myHook: InternalHookHandler = async (event) => {
  event.messages.push("Session saved to memory/2026-01-16-api-design.md");
};
```

这个机制让钩子不仅能执行后台操作，还能向用户反馈执行结果——虽然目前内置的钩子选择了静默运行（不推送用户可见的消息），但这个能力为自定义钩子提供了丰富的交互空间。

本节小结

1. **内部钩子**是 OpenClaw 的事件驱动扩展机制，基于发布/订阅模式实现，处理器注册到事件键上，事件触发时按注册顺序依次调用。
2. **钩子加载**分两阶段：目录发现（新系统）和配置处理器（遗留兼容），从 extra/bundled/managed/workspace 四个源加载并按优先级合并。
3. **钩子安装**支持本地路径、归档文件和 npm 包三种来源，内置路径穿越防护和备份回滚机制。
4. **四个内置钩子**覆盖了启动引导（boot-md）、会话记忆（session-memory）、命令审计（command-logger）和人格替换（soul-evil）四个场景。
5. **`agent:bootstrap` 事件**是最强大的扩展点，允许钩子在系统提示词构建前修改引导文件列表，实现对 Agent 行为的动态控制。
6. **事件模型**采用 type + action 二维键，支持类型级和动作级两种粒度的订阅，并通过 try/catch 实现错误隔离，保证系统稳定性。